

Projektplan mit Zeitschätzung

CVPortal – Webbasiertes Lebenslauf- und Profilmanagementsystem

Gesamtaufwand: ca. 98 Stunden **Reiner Programmieraufwand:** ca. 71 Stunden **Testaufwand:** ca. 13 Stunden **Dokumentation:** ca. 15 Stunden

Meilensteinplan

#	Meilenstein	Woche	Ergebnis
M1	Anforderungen abgeschlossen, Architektur festgelegt	1	Pflichtenheft, Datenbankmodell, API-Design
M2	Backend-Grundlage: Auth, DB, CORS lauffähig	2	JWT-Login funktioniert, CORS verifiziert
M3	Backend vollständig: alle REST-Endpunkte	3–4	API vollständig, via REST-Client testbar
M4	Frontend vollständig: alle Seiten, API-Calls	5–6	Vollständige Web-App nutzbar
M5	Visitenkarte + QR-Code umgesetzt	6	Öffentliche Karte mit QR-Code abrufbar
M6	Tests abgeschlossen, alle Bugs behoben	7	Alle 12 Testfälle bestanden
M7	Dokumentation und Abgabe	8	Vollständige Abgabe beider Projekte

Projektstrukturplan (PSP)

Phase 1 – Planung und Architekturdesign (13 h)

Aufgabe	Stunden
Anforderungsanalyse, Pflichtenheft verfassen	6 h
ER-Diagramm und Datenbankmodell entwerfen	2 h
REST-API Design: Endpunkte, Request/Response-DTOs definieren	2 h
Projektplan erstellen	2 h
Beide Maven-Projekte aufsetzen, pom.xml + Dependencies	1 h
Summe Phase 1	13 h

Phase 2 – Backend: Basis-Infrastruktur (14 h)

Aufgabe	Stunden
Alle Entities anlegen (User , CurriculumVitae , WorkExperience , Education , Skill , Language)	3 h

Aufgabe	Stunden
JPA-Repositories für alle Entities	1 h
H2-Datenbankverbindung + <code>data.sql</code> mit Testdaten	1 h
Spring Security + JWT: <code>JwtService</code> , <code>JwtAuthFilter</code> , <code>SecurityConfig</code>	5 h
<code>AuthController</code> (Register, Login) + <code>AuthService</code>	2 h
CORS-Konfiguration (<code>CorsConfig</code> , verifiziert mit Browser DevTools)	2 h
Summe Phase 2	14 h

Phase 3 – Backend: Fachliche REST-Endpunkte (12 h)

Aufgabe	Stunden
<code>CvController</code> + <code>CvService</code> : GET/PUT Stammdaten, Sichtbarkeit	2 h
<code>WorkExperienceController</code> + Service: CRUD für Berufserfahrung	2 h
<code>EducationController</code> + Service: CRUD für Ausbildung	1 h
<code>SkillController</code> + <code>LanguageController</code> : CRUD	1 h
<code>CardController</code> : öffentliche Endpunkte Visitenkarte + CV	2 h
<code>AdminController</code> : alle Teilnehmer abrufen, Benutzerverwaltung	2 h
<code>GlobalExceptionHandler</code> (strukturierte Fehler-Responses)	2 h
Summe Phase 3	12 h

Phase 4 – Frontend: Infrastruktur und Auth (5 h)

Aufgabe	Stunden
<code>ApiClientService</code> : RestTemplate konfigurieren, JWT-Header setzen	2 h
Login- und Registrierungsseite + <code>PageController</code>	2 h
JWT im SessionStorage speichern; automatische Weiterleitung bei abgelaufenem Token	1 h
Summe Phase 4	5 h

Phase 5 – Frontend: Lebenslauf-Bearbeitung (14 h)

Aufgabe	Stunden
Layout-Template (Navbar, Sidebar-Navigation, Footer)	2 h
Dashboard-Seite (CV-Vollständigkeitsanzeige, Schnelllinks)	2 h
Stammdaten-Formular	2 h
Berufserfahrungs-Verwaltung (Liste + Anlegen/Bearbeiten-Modal)	2 h

Aufgabe	Stunden
Ausbildungs-Verwaltung	2 h
Kenntnisse und Sprachen-Verwaltung	2 h
Sichtbarkeits-Toggle (öffentlich / privat)	2 h
Summe Phase 5	14 h

Phase 6 – Frontend: Öffentliche Ansichten (6 h)

Aufgabe	Stunden
Öffentliche Lebenslauf-Seite (/cv/{username}) – druckfreundlich	2 h
Web-Visitenkarte (/card/{username}) – visuelles Design, Karten-Optik	2 h
Admin-Übersicht (Teilnehmertabelle)	2 h
Summe Phase 6	6 h

Phase 7 – Wunschkriterien (6 h)

Aufgabe	Stunden
QR-Code-Generierung via ZXing im Backend + Anzeige auf Visitenkarte (W01)	3 h
Admin-Dashboard mit Statistiken (W04)	2 h
Passwort ändern im eigenen Profil	1 h
Summe Phase 7	6 h

Phase 8 – Testing (13 h)

Aufgabe	Stunden
Unit Tests: JwtService (Token generieren, validieren, abgelaufen)	2 h
Unit Tests: CvService (Vollständigkeitsgrad-Berechnung)	2 h
Unit Tests: AuthService (Registrierung, Login, Fehlerfall)	2 h
Integration Tests: AuthController mit MockMvc	2 h
Integration Tests: CvController mit MockMvc + JWT	2 h
Integration Tests: CORS-Header (OPTIONS-Request)	1 h
Manueller Test aller 12 Testfälle aus dem Pflichtenheft	2 h
Summe Phase 8	13 h

Phase 9 – Dokumentation und Abschluss (15 h)

Aufgabe	Stunden
Javadoc für alle public Methoden (Backend)	3 h
README.md Backend: API-Dokumentation, Startanleitung, Testbenutzer	2 h
README.md Frontend: Startanleitung, CORS-Erklärung	2 h
Kurzanleitung für die 3 Benutzerrollen	2 h
CORS-Erklärungsdokument (für Prüfungspräsentation)	1 h
Präsentationsvorbereitung (Folien, Live-Demo-Ablauf)	3 h
Finaler Code-Review, Bugfixes, Abgabe vorbereiten	2 h
Summe Phase 9	15 h

Gesamtübersicht

Phase	Bezeichnung	Stunden
1	Planung und Architekturdesign	13 h
2	Backend: Basis-Infrastruktur	14 h
3	Backend: Fachliche REST-Endpunkte	12 h
4	Frontend: Infrastruktur und Auth	5 h
5	Frontend: Lebenslauf-Bearbeitung	14 h
6	Frontend: Öffentliche Ansichten	6 h
7	Wunschkriterien	6 h
8	Testing	13 h
9	Dokumentation und Abschluss	15 h
Gesamt		98 h

Reiner Programmieraufwand (Phase 1 Coding-Anteil + Phasen 2–7): ~71 h (Mindestanforderung: 50 h)

Zeitlicher Ablauf (8 Wochen)

Woche 1:		Phase 1 – Planung (13h)
Woche 2:		Phase 2 – Backend Basis (14h)
Woche 3:		Phase 3 – Backend API, Teil 1 (12h)
Woche 4:		Phase 4 – Infrastruktur und Auth (5h) + Phase 5 Part 1 (7h)
Woche 5:		Phase 5 – Frontend CV-Bearbeitung Part 2 (7h) + Phase 6 Öffentliche Ansichten (6h)
Woche 6:		Phase 7 Wunschkriterien (6h) + Phase 8 Testing Part 1 (6h)

Woche 7: 
 Woche 8: 

Phase 8 – Testing (7h) + Phase 9 Doku (5h)
 Phase 9 – Doku + Abschluss (10h)

Risiken und Gegenmaßnahmen

Risiko	Wahrscheinlichkeit	Gegenmaßnahme
Spring Security/JWT komplexer als geplant	hoch	Spring Security Docs + Beispielprojekte vorab studieren; früh (Phase 2) beginnen
CORS-Probleme im Browser	mittel	CORS-Config früh testen (Ende Phase 2)
Frontend-Backend-Integration schlägt fehl	niedrig	API zuerst isoliert mit REST-Client (Postman) testen
Zeitüberschreitung beim Frontend-Design	mittel	Wunschkriterien (W02 PDF-Export) als letztes; Puffer in Phase 9
Zwei laufende Apps erhöhen Entwicklungskomplexität	mittel	Klare Port-Trennung; beide Apps per Skript oder als Docker Container starten
H2-Datei-Lock bei Parallelzugriff	niedrig	H2 im Auto-Mixed-Mode betreiben; Backend nur einmal starten
Benutzername mit Sonderzeichen bricht URL-Routing (/cv/{username})	niedrig	Whitelist für Benutzernamen (nur A-Z, 0-9, Bindestrich); URL-Encoding sicherstellen
PDF-Export (W02) unterschätzt	mittel	Flying Saucer rendert HTML nur eingeschränkt → vorab Proof-of-Concept; ggf. auf Alternativen ausweichen
Stunden-Puffer zu knapp	mittel	Phase 9 (Doku) hat 15h – bei Verzug Wunschkriterien W02/W04 streichen